

GraphIDS Dinâmico

Detecção de Intrusão em Topologias de Rede Instáveis

Feature: Grafos Dinâmicos & Inferência em Streaming

GraphIDS + DynamicGraph + StreamingDetector

O Problema

O GraphIDS original constrói um **grafo estático único** para todo o dataset.

Limitações em redes reais:

- Novos IPs aparecem constantemente

Dispositivos móveis, IoT, cloud elástico

- Arestas expiram

Fluxos têm tempo de vida finito

- Nós saem da rede

IPs deixam de ser usados

- Detecção offline

Processamento em lote, não em tempo real

Exemplo: Rede doméstica com 50 dispositivos entrando e saindo o tempo todo.

Arquitetura da Solução

Quatro pilares:

1. **DynamicGraph** — Estrutura de dados que gerencia a evolução do grafo

`add_edge()` $O(1)$, `remove_stale_edges()`, snapshots PyG

2. **SAGELayer Aprimorado** — Pesos temporais + propagação seletiva + cold start

3. **StreamingDetector** — Detecção fluxo-a-fluxo com janela deslizando

Transformer AE reconstrói embedding → erro = anomalia

4. **Treinamento Online** — Experience replay evita esquecimento catastrófico

Pipeline: Fluxo → DynamicGraph → GNN → Buffer → Transformer → Score → Alerta

DynamicGraph — O Coração da Feature

Mapeamento Dinâmico IP → node_id

IPs não vistos antes ganham ID automaticamente (incremento monotônico)

Armazenagem Temporal

Cada aresta guarda timestamp de criação e flag de atividade (soft delete)

Expiração & Remoção

`remove_stale_edges()` → arestas > TTL são removidas em $O(E)$

`remove_inactive_nodes()` → nós sem atividade são limpos

Backward Compatibility

`get_current_graph()` → Data PyG = mesmo formato do modelo original

Decisão: Listas paralelas (`_edge_src`, `_edge_dst`, `_edge_attr...`)

`add_edge` $O(1)$ amortizado, sem realocação de tensores

Adaptações no Modelo GNN

Pesos Temporais

`temporal_weights` multiplica `edge_attr` no `MessagePassing` → arestas antigas contribuem menos

Peso típico: $\exp(-\lambda \times \text{idade})$, λ calibrado para meia-vida de ~10min

Propagação Seletiva

`node_mask` → propaga apenas em nós afetados por mudanças recentes

ColdStartNodeInitializer

Nós novos (sem histórico) → `embedding` = média dos vizinhos

Se sem vizinhos → `embedding default` aprendido (parâmetro treinável)

Backward Compat: parâmetros novos `default=None`

Código existente (`train.py`, `test.py`) continua funcionando sem alterações

StreamingDetector — Inferência em Tempo Real

process_flow() — Um fluxo por vez:

add_edge() → encoder GNN → SlidingWindowBuffer → Transformer AE
→ score

process_batch() — Lote otimizado:

1 forward GNN + 1 forward Transformer AE para N fluxos

Stack de janelas: [N, window_size, dim] → latência ~N vezes menor

Threshold Adaptativo

MAD (median + multiplier × MAD) sobre janela de scores recentes

Ou validation_f1 se houver labels disponíveis

Thread Safety: Locks no buffer, threshold, e process_lock para atomicidade

Treinamento Online & Experience Replay

`train_online()`

Fine-tuning rápido em novos dados sem retreino completo

Loss combinada: $\text{loss_dados_novos} + \text{loss_replay_buffer}$ (amostras antigas)

`train_temporal_windows()`

Itera sobre janelas temporais do StreamingNetFlowDataset

Treina janela i , valida janela $i+1$, adiciona i ao replay buffer

`update_threshold_online()`

Threshold com EMA smoothing: $\alpha \times \text{novo} + (1-\alpha) \times \text{anterior}$

$\alpha=0.1 \rightarrow$ mudanças graduais, evita oscilações bruscas

Benefício: Modelo se adapta a novos padrões sem esquecer ataques passados

Fluxo de Execução (--streaming)

Terminal:

```
python main.py --dataset NF-UNSW-NB15-v3 --streaming
```

Passo a passo:

1. StreamingNetFlowDataset carrega dados em N janelas temporais
2. Treina modelo nas primeiras 60% das janelas
3. Para cada janela restante:
 - a. Executa inferência (encoder + transformer AE)
 - b. Calcula F1 e PR-AUC
 - c. Atualiza threshold com EMA
 - d. Loga no wandb
4. Retorna modelo adaptado ao novo domínio temporal

Resultados & Métricas

Testes de Integração:

DynamicGraph + SAGELayer + StreamingDetector + Threshold → todos passam

Regressão:

15/15 testes de config/preprocessing passam (sem alterações)

4/9 testes de training passam (5 falham por GLIBC_2.33, problema de ambiente)

Backward compatibility mantida — parâmetros novos opcionais

Eficiência por operação:

add_edge $O(1)$ | Snapshot $O(E+N)$ | GNN Forward $O(E+N)$

Transformer AE $O(W^2)$ — gargalo identificado, $W=512$ de default

Gargalo: Transformer Autoencoder — 262K operações de atenção por janela

Mitigação: reduzir window_size, usar process_batch, stride < window

Considerações de Segurança

Cold Start: Novos nós sem histórico → embedding inicial pode gerar falsos positivos

Mitigação: inicialização por média dos vizinhos + embedding default aprendido

Graph Poisoning: Atacante conecta nós maliciosos a muitos legítimos

Mitigação: max_nodes (100K), grau máximo implícito, monitoramento de componentes

Threshold Drift: Scores podem mudar gradualmente com o comportamento da rede

Mitigação: EMA ($\alpha=0.1$) suaviza mudanças; recalibração periódica

Flapping (Oscilação): Nós entrando e saindo rapidamente

Mitigação: TTL com histerese — nó só removido após inatividade prolongada

Limitação de Memória: max_nodes=100K evita DoS por inundação de IPs

Decisões Arquiteturais

1. **DynamicGraph centralizado** — gestão de topologia separada do modelo
2. **Soft delete (`_edge_active`)** — remoção $O(1)$, sem fragmentação
3. **Snapshots com remapeamento** — nós removidos não criam buracos nos tensores
4. **Pesos temporais no MessagePassing** — integração natural com o PyG
5. **Thread safety** — locks para inferência concorrente
6. **Experience replay** — mais simples que EWC, previne forgetting
7. **Threshold com EMA** — mudanças graduais sem oscilações

Obrigado!

Dynamic GraphIDS — Próximos Passos

1. Integração com Kafka/Fila de mensagens para dados reais
2. Testes com datasets reais (CIC-IDS2018, UNSW-NB15)
3. Benchmark de latência vs. acurácia em diferentes window_sizes
4. Deploy contínuo com retreinamento periódico automático

Repositório: `/home/marcos/GraphIDS-repo`

Documentação completa em docs/